

Edna Elveus

Problem 1: Quadratic Equations

Source code (p1_Elveus_Edna.py):

```
"""Solves quadratic equations and plots them."""

from matplotlib import pyplot as plt

while True:
    a = (input("Enter a: "))
    if a == "":
        break

    b = float(input("Enter b: "))
    c = float(input("Enter c: "))
    a = float(a)

    determinant = b**2 - 4*a*c

    if determinant < 0:
        print("no real solutions")
    elif determinant == 0:
        x1 = -b / (2*a)
        print(f"one solution: {x1:.5f}")
    else:
        x1 = (-b - determinant**0.5) / (2*a)
        x2 = (-b + determinant**0.5) / (2*a)
        print(f"two solutions: x1={x1:.5f} x2={x2:.5f}")

    if determinant > 0:
        root_min = min(x1, x2)
        root_max = max(x1, x2)

        margin = (root_max - root_min) + 1

        x_min = root_min - margin
        x_max = root_max + margin
    elif determinant == 0:
        x_opt = -b / (2*a)
```

```

margin = 2
x_min = x_opt - margin
x_max = x_opt + margin
else:
    x_opt = -b / (2*a)
    margin = 5
    x_min = x_opt - margin
    x_max = x_opt + margin

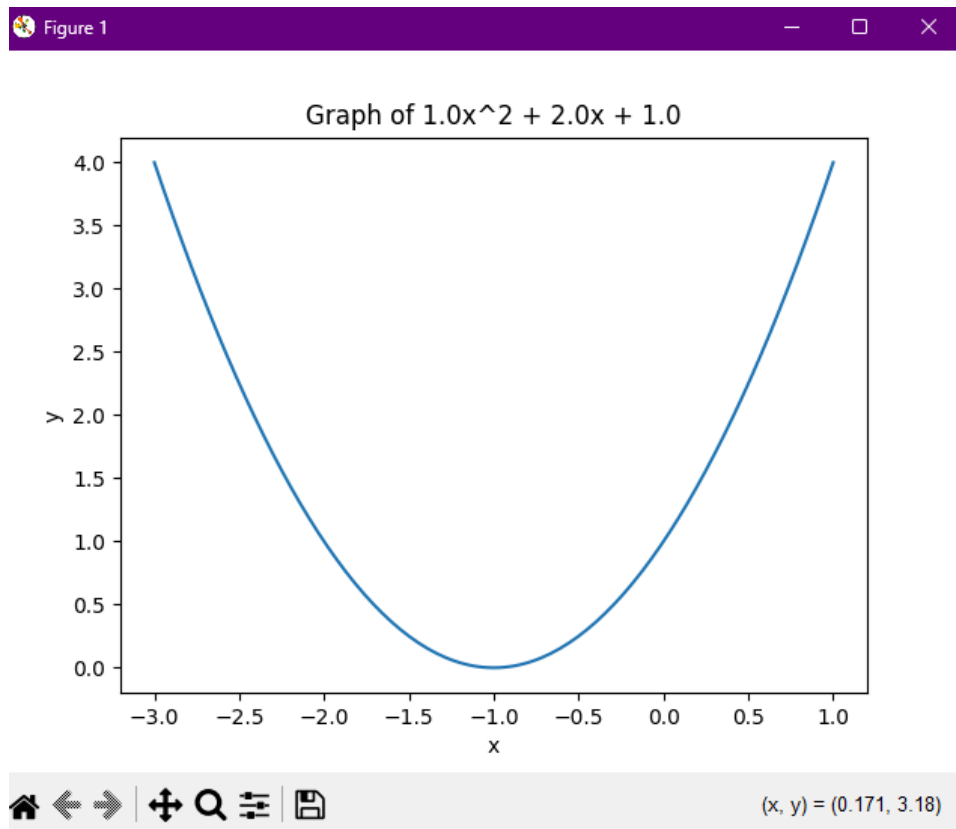
points = 150
step = (x_max - x_min) / (points - 1)
xs = [x_min + i * step for i in range(points)]

ys = [a * x**2 + b * x + c for x in xs]

plt.plot(xs, ys)
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"Graph of {a}x^2 + {b}x + {c}")
plt.show()

```

Screenshot - Matplotlib figure window



Problem 2: Pythagorean Numbers

Source code (p2_Elveus_Edna.py):

```
"""
Pythagorean Triple Finder
Problem 2 - CS Assignment
"""

def find_Pythagorean(n):
    """
    Find all Pythagorean triples (a, b, c) where  $0 < a, b, c \leq n$ .

    Uses a brute-force approach with three nested loops checking all
    possible combinations. This intentionally produces both (3,4,5) and
    (4,3,5) as separate triples since they represent different
    assignments
    to a and b, which is consistent with a pure brute-force
    interpretation.

    Args:
        n (int): Upper bound for values of a, b, and c

    Returns:
        list: List of tuples (a, b, c) where  $a^2 + b^2 = c^2$ 
    """
    triples = []

    # Brute-force: check all possible values for a, b, and c
    for a in range(1, n + 1):
        for b in range(1, n + 1):
            for c in range(1, n + 1):
                if a**2 + b**2 == c**2:
                    triples.append((a, b, c))

    return triples

def main():
    """
    Main program: reads n, finds Pythagorean triples, and displays
    them.
    """
```

```

# Read positive integer n from terminal
n = int(input("Enter a positive integer n: "))

# Ensure n is positive
if n <= 0:
    print("Please enter a positive integer.")
    return

# Find all Pythagorean triples
triples = find_Pythagorean(n)

# Display the resulting triples to the terminal
if triples:
    print(f"\nFound {len(triples)} Pythagorean triple(s) for n = {n}:")
    for triple in triples:
        # Print in a clean format: (a, b, c)
        print(f"({triple[0]}, {triple[1]}, {triple[2]})")
else:
    print(f"\nNo Pythagorean triples found for n = {n}.")

# Run the program
if __name__ == "__main__":
    main()

```

Problem 3: Duplicated Substrings

Source code (p3_Elveus_Edna.py):

```

"""
Duplicate Substring Finder
Problem 3 - CS Assignment
"""

def find_dup_str(s, n):
    """
    Find the first-occurring substring of length n that appears at
    least twice
    non-overlapping in string s.
    """

```

Uses a brute-force nested loop approach with string slicing and plain

equality comparisons (no `str.find()`).

Args:

`s (str)`: The string to search in

`n (int)`: Length of substring to find

Returns:

`str`: The first duplicated substring of length `n`, or `""` if none exists

```
"""
```

```
if n <= 0 or len(s) < 2 * n:
```

```
    return ""
```

```
# Slide window across the string
```

```
for i in range(len(s) - n + 1):
```

```
    candidate = s[i:i+n]
```

```
    # Check for non-overlapping duplicate starting at position j
```

```
    # j must be at least i + n to ensure non-overlapping
```

```
    for j in range(i + n, len(s) - n + 1):
```

```
        if s[j:j+n] == candidate:
```

```
            return candidate
```

```
return ""
```

```
def find_max_dup(s):
```

```
    """
```

```
    Find the longest duplicated substring in string s.
```

Starts from the maximum possible length (`len(s)//2`) and works downward,

using `find_dup_str` internally to check each length.

Args:

`s (str)`: The string to search in

Returns:

`str`: The longest duplicated substring, or `""` if none exists

```
"""
```

```
# Maximum possible length for a non-overlapping duplicate
```

```

max_len = len(s) // 2

# Try from largest to smallest length
for n in range(max_len, 0, -1):
    result = find_dup_str(s, n)
    if result != "":
        return result

return ""

def test_find_dup_str():
    """
    Test function for find_dup_str - reads s and n from terminal.
    """
    print("\n--- Testing find_dup_str ---")
    s = input("Enter string s: ")
    n = int(input("Enter length n: "))

    result = find_dup_str(s, n)
    print(f"Result: '{result}'")

def test_find_max_dup():
    """
    Test function for find_max_dup - reads s from terminal.
    """
    print("\n--- Testing find_max_dup ---")
    s = input("Enter string s: ")

    result = find_max_dup(s)
    print(f"Longest duplicated substring: '{result}'")

def main():
    """
    Main program: runs both test functions.
    """
    test_find_dup_str()
    test_find_max_dup()

# Run the program

```

```
if __name__ == "__main__":  
    main()
```

Problem 4: Function Visualization

Source code (p4_Elveus_Edna.py):

```
"""  
Function Plotter  
Problem 4 - CS Assignment  
"""  
  
import math  
import matplotlib.pyplot as plt  
  
def plot_function(fun_str, domain, ns):  
    """  
    Plot a mathematical function defined by a string expression.  
  
    Computes ns evenly spaced points across the domain, evaluates the  
    function at each point using eval(), displays a table of values,  
    and generates a plot of the function.  
  
    Args:  
        fun_str (str): String expression like "2 * x + 3" or  
        "math.sin(x)"  
        domain (tuple): (xmin, xmax) defining the range  
        ns (int): Number of sample points to use  
    """  
    xmin, xmax = domain  
  
    # Generate ns evenly spaced points across [xmin, xmax]  
    if ns > 1:  
        step = (xmax - xmin) / (ns - 1)  
    else:  
        step = 0  
  
    xs = []  
    for i in range(ns):  
        xs.append(xmin + i * step)  
  
    # Evaluate function at each x using eval()
```

```

ys = []
for x in xs:
    # eval() will see x in the local scope
    y = eval(fun_str)
    ys.append(y)

# Display table of values
print("      x      y")
for i in range(ns):
    # Format: 4 decimal places with + sign for positive values
    print(f"{xs[i]:+8.4f} {ys[i]:+8.4f}")

# Plot the function
plt.plot(xs, ys)
plt.xlabel("x")
plt.ylabel("y")
plt.title(f"Function: {fun_str}")
plt.grid(True, alpha=0.3)
plt.show()

def main():
    """
    Main program: reads function string, domain, and number of samples,
    then calls plot_function.
    """
    print("Function Plotter")
    print("-" * 40)

    # Read inputs from terminal
    fun_str = input("Enter function (e.g., '2 * x + 3'): ")
    xmin = float(input("Enter xmin: "))
    xmax = float(input("Enter xmax: "))
    ns = int(input("Enter number of samples: "))

    # Validate inputs
    if ns <= 0:
        print("Number of samples must be positive.")
        return

    if xmin >= xmax:
        print("xmin must be less than xmax.")
        return

```

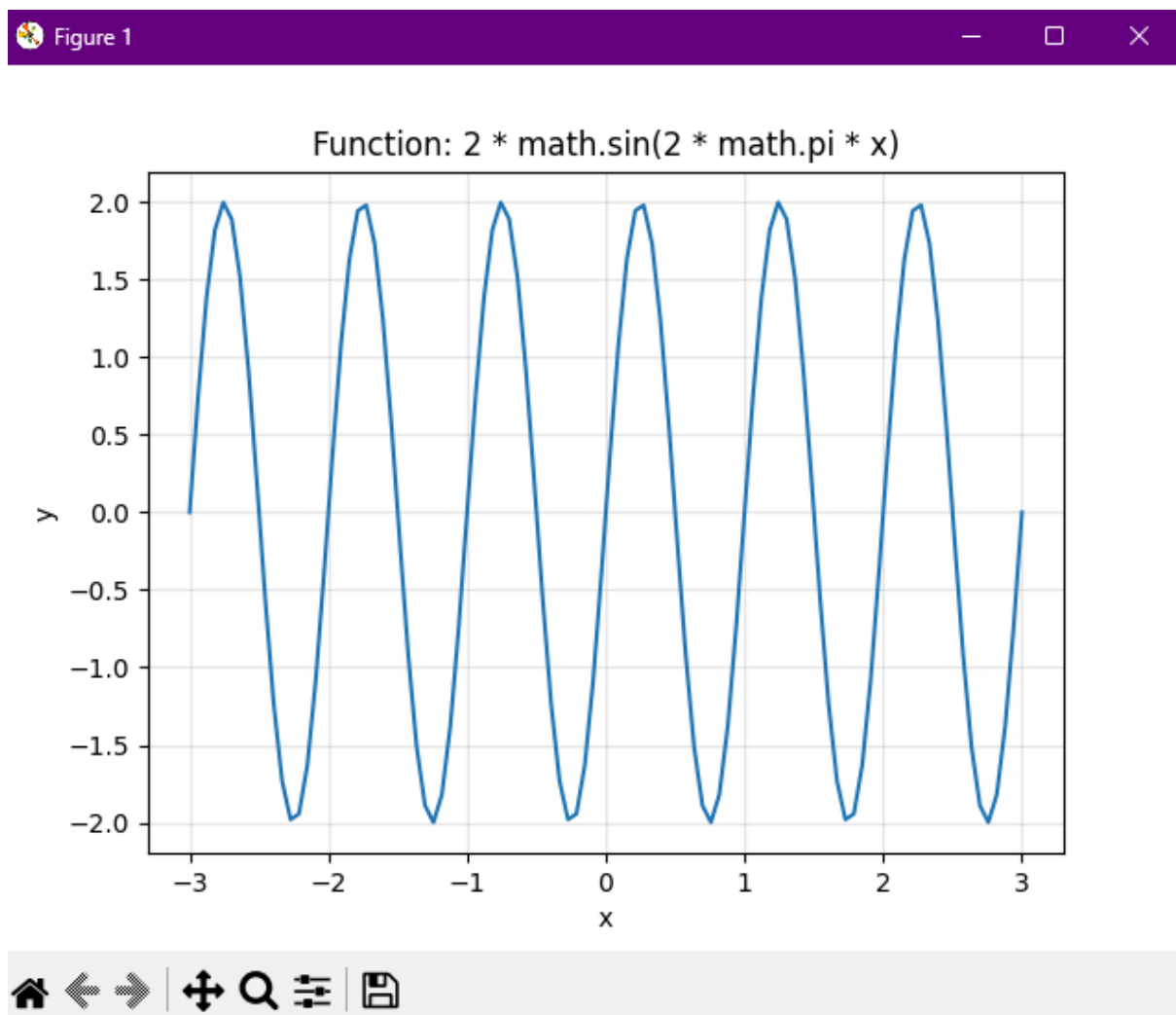
```

# Call plot_function
plot_function(fun_str, domain=(xmin, xmax), ns=ns)

# Run the program
if __name__ == "__main__":
    main()

```

Screenshot - Matplotlib figure window



Problem 5: Interactive Caesar Ciper with coding agent help

Source code (p5_Elveus_Edna.py):

```
"""
```

Caesar Cipher with Frequency Analysis

Problem 5 - CS Assignment

"""

```
def caesar_cipher(text, shift):
    """
    Encrypt text using Caesar cipher with the given shift.

    Shifts each letter by shift positions, preserving spaces,
    punctuation,
    and original casing. Uses string indexing and loops for mechanics.

    Args:
        text (str): The plaintext to encrypt
        shift (int): Number of positions to shift each letter

    Returns:
        str: The encrypted text
    """
    result = []

    for i in range(len(text)):
        char = text[i]

        # Check if character is a letter
        if 'a' <= char <= 'z':
            # Lowercase letter
            base = ord('a')
            position = ord(char) - base
            new_position = (position + shift) % 26
            result.append(chr(base + new_position))
        elif 'A' <= char <= 'Z':
            # Uppercase letter
            base = ord('A')
            position = ord(char) - base
            new_position = (position + shift) % 26
            result.append(chr(base + new_position))
        else:
            # Non-letter: preserve as-is
            result.append(char)

    return ''.join(result)
```

```

def caesar_decipher(ciphertext, shift):
    """
    Decrypt text encrypted with Caesar cipher.

    Simply calls caesar_cipher with negative shift.

    Args:
        ciphertext (str): The encrypted text to decrypt
        shift (int): The shift used for encryption

    Returns:
        str: The decrypted text
    """
    return caesar_cipher(ciphertext, -shift)


def letter_frequency(text):
    """
    Count occurrences of each letter a-z in the text.

    Case-insensitive (treats 'A' and 'a' as the same).
    Ignores non-alphabetic characters entirely.

    Args:
        text (str): The text to analyze

    Returns:
        dict: Dictionary mapping letters (a-z) to their counts
    """
    # Initialize dictionary with all letters a-z set to 0
    frequency = {}
    for i in range(26):
        letter = chr(ord('a') + i)
        frequency[letter] = 0

    # Count letters (case-insensitive, ignore non-letters)
    for char in text:
        if 'a' <= char <= 'z':
            frequency[char] += 1
        elif 'A' <= char <= 'Z':
            # Convert to lowercase for consistency

```

```

        lower_char = chr(ord('a') + (ord(char) - ord('A')))
        frequency[lower_char] += 1

    return frequency

def display_frequency(freq):
    """
    Display letter frequency dictionary in a readable format.

    Args:
        freq (dict): Dictionary of letter counts
    """
    print("\nLetter Frequency:")
    for letter in sorted(freq.keys()):
        if freq[letter] > 0:
            print(f"    {letter}: {freq[letter]}")

def main():
    """
    Main program with terminal menu for Caesar cipher operations.
    """
    print("Caesar Cipher Program")
    print("=" * 40)

    while True:
        print("\nOptions:")
        print("    1. Encrypt a message")
        print("    2. Decrypt a message")
        print("    3. Quit")

        choice = input("Enter your choice (1-3): ")

        if choice == '3':
            print("Goodbye!")
            break

        if choice == '1':
            # Encryption mode
            message = input("Enter the message to encrypt: ")
            shift = int(input("Enter shift value: "))

```

```

    # Encrypt
    ciphertext = caesar_cipher(message, shift)
    print(f"\nOriginal: {message}")
    print(f"Encrypted: {ciphertext}")

    # Show frequency analysis
    print("\n--- Frequency Analysis ---")
    freq = letter_frequency(ciphertext)
    display_frequency(freq)

    # Show decryption
    decrypted = caesar_decipher(ciphertext, shift)
    print(f"\nDecrypted (verification): {decrypted}")

elif choice == '2':
    # Decryption mode
    ciphertext = input("Enter the message to decrypt: ")
    shift = int(input("Enter shift value: "))

    # Decrypt
    plaintext = caesar_decipher(ciphertext, shift)
    print(f"\nCiphertext: {ciphertext}")
    print(f"Decrypted: {plaintext}")

    # Show frequency analysis of ciphertext
    print("\n--- Frequency Analysis ---")
    freq = letter_frequency(ciphertext)
    display_frequency(freq)

else:
    print("Invalid choice. Please enter 1, 2, or 3.")

# Run the program
if __name__ == "__main__":
    main()

```

Source code (p5_Elveus_Edna_test.py):

```

"""
Unit tests for Caesar Cipher functions
Problem 5 - CS Assignment
"""

import unittest

```

```
from p5_Elveus_Edna import caesar_cipher, caesar_decipher,
letter_frequency
```

```
class TestCaesarCipher(unittest.TestCase):
    """Test cases for Caesar cipher functions."""

    def test_caesar_cipher_basic(self):
        """Test basic encryption with positive shift."""
        self.assertEqual(caesar_cipher("abc", 1), "bcd")
        self.assertEqual(caesar_cipher("xyz", 1), "yza")
        self.assertEqual(caesar_cipher("ABC", 1), "BCD")
        self.assertEqual(caesar_cipher("XYZ", 1), "YZA")

    def test_caesar_cipher_negative_shift(self):
        """Test encryption with negative shift."""
        self.assertEqual(caesar_cipher("bcd", -1), "abc")
        self.assertEqual(caesar_cipher("yza", -1), "xyz")
        self.assertEqual(caesar_cipher("BCD", -1), "ABC")
        self.assertEqual(caesar_cipher("YZA", -1), "XYZ")

    def test_caesar_cipher_zero_shift(self):
        """Test encryption with zero shift (no change)."""
        text = "Hello World"
        self.assertEqual(caesar_cipher(text, 0), text)

    def test_caesar_cipher_case_preservation(self):
        """Test that case is preserved during encryption."""
        self.assertEqual(caesar_cipher("AbC", 1), "BcD")
        self.assertEqual(caesar_cipher("ZyX", 1), "AzY")

    def test_caesar_cipher_non_letters(self):
        """Test that spaces and punctuation are preserved."""
        text = "Hello, World!"
        encrypted = caesar_cipher(text, 1)
        self.assertEqual(encrypted, "Ifmmp, Xpsme!")
        self.assertEqual(encrypted[5], ',') # Comma preserved
        self.assertEqual(encrypted[6], ' ') # Space preserved
        self.assertEqual(encrypted[-1], '!') # Exclamation preserved

    def test_caesar_cipher_large_shift(self):
        """Test with large shift values (wrapping)."""
        self.assertEqual(caesar_cipher("abc", 26), "abc")
```

```

self.assertEqual(caesar_cipher("abc", 27), "bcd")
self.assertEqual(caesar_cipher("abc", -26), "abc")
self.assertEqual(caesar_cipher("abc", -27), "zab")

def test_caesar_decipher(self):
    """Test decryption function."""
    plaintext = "Hello World"
    shift = 5
    ciphertext = caesar_cipher(plaintext, shift)
    self.assertEqual(caesar_decipher(ciphertext, shift), plaintext)

def test_caesar_decipher_negative_shift(self):
    """Test decryption with negative shift."""
    plaintext = "Hello World"
    shift = -5
    ciphertext = caesar_cipher(plaintext, shift)
    self.assertEqual(caesar_decipher(ciphertext, shift), plaintext)

def test_letter_frequency_basic(self):
    """Test basic letter frequency counting."""
    freq = letter_frequency("abc")
    self.assertEqual(freq['a'], 1)
    self.assertEqual(freq['b'], 1)
    self.assertEqual(freq['c'], 1)
    self.assertEqual(freq['d'], 0)

def test_letter_frequency_case_insensitive(self):
    """Test that frequency counting is case-insensitive."""
    freq = letter_frequency("AaBb")
    self.assertEqual(freq['a'], 2)
    self.assertEqual(freq['b'], 2)
    self.assertEqual(freq['c'], 0)

def test_letter_frequency_ignore_non_letters(self):
    """Test that non-alphabetic characters are ignored."""
    freq = letter_frequency("a1b2c3!@#")
    self.assertEqual(freq['a'], 1)
    self.assertEqual(freq['b'], 1)
    self.assertEqual(freq['c'], 1)
    self.assertEqual(freq['d'], 0)

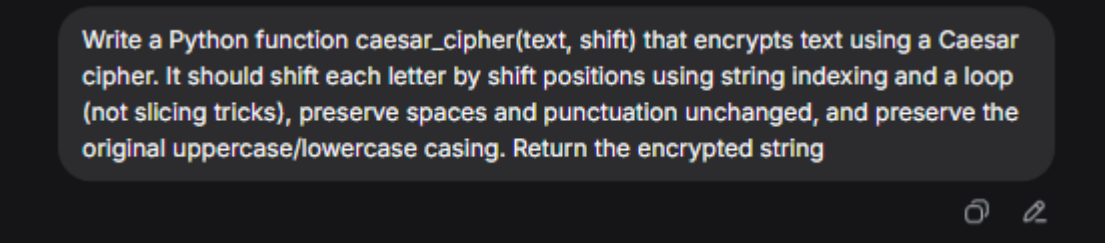
def test_letter_frequency_mixed_text(self):
    """Test frequency counting on mixed text."""

```

```
text = "Hello World"
freq = letter_frequency(text)
self.assertEqual(freq['h'], 1) # 'h' in 'Hello'
self.assertEqual(freq['e'], 1)
self.assertEqual(freq['l'], 3) # 'Hello' has 2, 'World' has 1
self.assertEqual(freq['o'], 2) # 'Hello' and 'World'
self.assertEqual(freq['w'], 1)
self.assertEqual(freq['r'], 1)
self.assertEqual(freq['d'], 1)

if __name__ == "__main__":
    unittest.main()
```

Coding agent interaction — Prompt 1:

A screenshot of a coding agent's prompt, displayed in a dark-themed interface. The prompt text is white and asks for a Python function to encrypt text using a Caesar cipher. The function signature is `caesar_cipher(text, shift)`. The requirements specify using string indexing and a loop, preserving spaces, punctuation, and case. The prompt is contained within a rounded rectangular box with a dark background. In the bottom right corner of the box, there are two small, faint icons: a square and a pencil.

Write a Python function `caesar_cipher(text, shift)` that encrypts text using a Caesar cipher. It should shift each letter by `shift` positions using string indexing and a loop (not slicing tricks), preserve spaces and punctuation unchanged, and preserve the original uppercase/lowercase casing. Return the encrypted string

Coding agent interaction — Response 1:

```
python Copy Download

def caesar_cipher(text, shift):
    """
    Encrypt text using Caesar cipher with the given shift.

    Shifts each letter by shift positions, preserving spaces, punctuation,
    and original casing. Uses string indexing and loops for mechanics.

    Args:
        text (str): The plaintext to encrypt
        shift (int): Number of positions to shift each letter

    Returns:
        str: The encrypted text
    """
    result = []

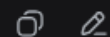
    for i in range(len(text)):
        char = text[i]

        # Check if character is a letter
        if 'a' <= char <= 'z':
            # Lowercase letter
            base = ord('a')
            position = ord(char) - base
            new_position = (position + shift) % 26
            result.append(chr(base + new_position))
        elif 'A' <= char <= 'Z':
            # Uppercase letter
            base = ord('A')
            position = ord(char) - base
            new_position = (position + shift) % 26
            result.append(chr(base + new_position))
        else:
            # Non-letter: preserve as-is
            result.append(char)

    return ''.join(result)
```

Coding agent interaction — Prompt 2:

Write a function `letter_frequency(text)` that counts how many times each letter of the alphabet appears in a string, ignoring case and skipping any non-alphabetic characters. Return the counts as a dictionary with all 26 letters as keys, initialized to 0 even if a letter doesn't appear.



Coding agent interaction — Response 2:

```
python Copy Download

def letter_frequency(text):
    """
    Count occurrences of each letter a-z in the text.

    Case-insensitive (treats 'A' and 'a' as the same).
    Ignores non-alphabetic characters entirely.

    Args:
        text (str): The text to analyze

    Returns:
        dict: Dictionary mapping letters (a-z) to their counts
    """
    # Initialize dictionary with all letters a-z set to 0
    frequency = {}
    for i in range(26):
        letter = chr(ord('a') + i)
        frequency[letter] = 0

    # Count letters (case-insensitive, ignore non-letters)
    for char in text:
        if 'a' <= char <= 'z':
            frequency[char] += 1
        elif 'A' <= char <= 'Z':
            # Convert to lowercase for consistency
            lower_char = chr(ord('a') + (ord(char) - ord('A')))
            frequency[lower_char] += 1

    return frequency
```

Coding agent interaction — Prompt 3:

Write a Python function `caesar_decipher(ciphertext, shift)` that decrypts text that was encrypted with a Caesar cipher using the given shift. It should return the original plaintext string. Reuse `caesar_cipher` internally rather than reimplementing the shift logic.

Coding agent interaction — Response 3:

python

 Copy  Download

```
def caesar_decipher(ciphertext, shift):  
    """  
    Decrypt text encrypted with Caesar cipher.  
  
    Simply calls caesar_cipher with negative shift.  
  
    Args:  
        ciphertext (str): The encrypted text to decrypt  
        shift (int): The shift used for encryption  
  
    Returns:  
        str: The decrypted text  
    """  
    return caesar_cipher(ciphertext, -shift)
```

